

Informe de Exposiciones

4to Software "A"

Alberto Jeanpool Marcillo Ponce.

Repositorio individual GitHub:

<https://github.com/poolgen/Informe-de-la-exposicion-MarcilloPonce>

Grupo A: ReqView

Mesias Jhon: Empezó explicando qué es ReqView y en qué se diferencia de simplemente usar Word o Excel para manejar requerimientos. Algo que me quedó bastante claro es que la herramienta trata cada requerimiento como algo propio, con su ID, su estado, su prioridad y un historial de todo lo que se le ha cambiado. También tocó el tema de la trazabilidad, es decir que uno puede ver cómo una necesidad del cliente termina conectada con un requerimiento, y ese requerimiento con su caso de prueba. Cerró diciendo que el documento de especificación se genera solo, sin que uno tenga que estar copiando y pegando nada.

Práctica: Abrió el documento ERS que ya tenían armado, se fue a la sección de Necesidades del Stakeholder y empezó a agregar las necesidades como objetos dentro de esa sección. Después hizo lo mismo con los Requisitos Funcionales y No Funcionales, para que se viera cómo queda todo organizado.

Mera Erick: Su parte fue sobre cómo se relaciona ReqView con la Ingeniería de Requisitos. Lo que más me quedó fue que dejó bien claro que la herramienta ayuda, pero no piensa por uno, el analista sigue siendo el que interpreta las necesidades y decide si un requerimiento está bien o mal escrito. Habló también de las funciones principales, como los documentos estructurados, los atributos personalizados y el historial de cambios.

Práctica: Mostró cómo crear casos de prueba dentro de ReqView. Usó como ejemplo un requerimiento de registro de pagos y armó el caso de prueba para comprobar que el pago se guarda bien y queda vinculado al cliente.

María Escudero: Explicó cómo está organizado el trabajo dentro de la herramienta, primero el proyecto, luego los documentos, y dentro de esos los objetos, que pueden ser secciones, necesidades o requerimientos. También habló de las ventajas, como la trazabilidad y los filtros, y fue honesta con las limitaciones, por ejemplo que algunas funciones son de pago, que hay cosas que toca configurar a mano, y que si un requerimiento está mal redactado, ReqView no lo corrige solo.

Práctica: Agregó atributos a los requerimientos, como tipo, fuente, estado y prioridad MoSCoW, para que cada uno tenga su clasificación sin mezclarla con la descripción.

Díaz Steven: Cerró con una comparativa entre ReqView y trabajar con documentos normales, mostrando que la diferencia está en que cada requerimiento se maneja de forma independiente y se puede relacionar con otros elementos. Aplicó eso al PFC para mostrar cómo quedaría organizado en un proyecto real.

Práctica: Creó un proyecto desde cero y mostró cómo enlazar una necesidad con un requerimiento funcional, y ese requerimiento con un caso de prueba, todo dentro de la misma herramienta.

Grupo B: Jama Connect (Mi grupo)

Grupo C: ZenTao

Castro Ariel: Presentó ZenTao como una plataforma ágil cuyo nombre viene de Zen y Tao. Explicó que tiene módulos como Backlog, Kanban, Sprints, Wiki e Issues, y fue relacionando cada uno con las fases de la Ingeniería de Requisitos: el Backlog para capturar historias de usuario, la Wiki para documentar, los criterios de aceptación para validar, entre otros.

Práctica: Creó un programa en ZenTao y agregó historias de usuario, mostrando cómo funciona el flujo básico desde que se registra una necesidad.

Mora Alex: Hizo la comparativa entre ZenTao y otras herramientas conocidas, como Jira, Trello y Azure DevOps. Fue señalando en qué le gana a cada una y en qué se queda corta.

Práctica: Mostró paso a paso cómo crear un proyecto en ZenTao y cómo enlazarlo con un producto dentro de la plataforma.

Vera Anthony: Llevó ZenTao a un caso real, el módulo de IA de un Sistema de Mantenimiento de Palma Africana. Contó que lo usaron en tres momentos distintos del proyecto, al inicio para registrar las funcionalidades en el Backlog, después para documentar los requerimientos técnicos en la Wiki, y más adelante para actualizar el proceso cuando hubo cambios en la elicitación.

Práctica: Registró un producto en ZenTao y mostró los campos visibles: nombre, estado, fechas de inicio y fin, e historias de usuario asociadas.

Villafuerte Allan: Cerró la exposición explicando cómo los módulos de ZenTao se conectan con la Ingeniería de Requisitos como disciplina, y después presentó las conclusiones del grupo, juntando todo lo que habían explicado sus compañeros.

Práctica: Armó un Sprint completo en ZenTao bajo la metodología Scrum, desde la configuración hasta ver cómo se asignan y siguen las tareas.

Grupo D: Odoo

Rizzo Nagib: Arrancó explicando qué es Odoo, un sistema ERP que junta varias áreas de una empresa en un solo lugar, de forma modular. Después se fue a las funcionalidades más relevantes para proyectos, la gestión de tareas con vistas Kanban, lista y calendario, el manejo de dependencias e hitos para saber qué tarea bloquea a cuál, y el control de tiempos con Timesheets, que incluso se pueden llenar desde el celular. Al final habló de hacia dónde va Odoo, mencionando integración con IA y análisis predictivo, y dejó un análisis crítico, que complementa al analista, no lo reemplaza, y que sí es aplicable en Ecuador.

Práctica: Mostró cómo crear un proyecto desde cero en Odoo, configurar las etapas del tablero Kanban y asignar tareas para que aparezcan en el flujo de trabajo.

Crespo Obed: Explicó cómo Odoo conecta el área comercial con los proyectos, cuando un cliente aprueba una cotización, el sistema puede convertirla directamente en un proyecto. También habló de cómo se generan facturas según las horas trabajadas o los servicios prestados, y en la parte de colaboración mencionó el chat integrado, los comentarios y las notificaciones automáticas cuando cambia el estado de una tarea. Cerró con ventajas y limitaciones, que es muy completo y modular, pero algunas funciones avanzadas son de pago y hay una curva de aprendizaje al principio.

Práctica: Mostró cómo crear una encuesta en Odoo, configuró las opciones, probó distintos tipos de respuesta como texto libre, opción múltiple y matriz de selección, simuló la encuesta y revisó el resumen de resultados.

Amagua Robyn: Habló de las funcionalidades de Odoo aplicadas a un proyecto de Ingeniería de Requerimientos. Por ejemplo, con la gestión de tareas se pueden crear actividades como entrevistas a usuarios o elaboración de diagramas, y asignarlas a cada miembro del equipo. Con el control de tiempos, un analista puede registrar cuántas horas dedicó a cada actividad, lo que ayuda a ver si los tiempos planificados se están cumpliendo, y también mencionó la parte financiera, que si el proyecto tiene presupuesto definido, se pueden ir registrando los gastos para no pasarse. Al comparar con Jira y Azure DevOps, destacó que Odoo tiene versión gratuita y la ventaja de integrar proyectos con ventas, inventario y finanzas en un solo sistema.

Práctica: Creó la tarea "Monitoreo de calidad del agua", le asignó un responsable, le puso prioridad alta, escribió la descripción del requerimiento y agregó comentarios para ir informando avances.

Grupo E: Redmine + RE Plugin

Ponce Mery: Empezó contextualizando el problema que Redmine busca resolver, cuando los requerimientos cambian sin control nadie sabe qué cambió ni quién lo hizo, y la documentación termina dispersa en varios archivos. Luego explicó qué es Redmine, una herramienta web de código abierto que pueden usar varios usuarios a la vez según los permisos que tengan, y que funciona desde cualquier equipo conectado a la red. Describió sus funciones principales, gestión de proyectos y tareas, control de roles, cronogramas, calendarios y reportes de avance.

Práctica: Mostró cómo registrar usuarios en Redmine, organizarlos en grupos de trabajo y asignarles perfiles con permisos específicos según lo que les toca hacer en el proyecto.

Gamarra Edhú: Presentó el Redmine RE Plugin como un complemento que le agrega a Redmine las funciones específicas para Ingeniería de Requisitos. Con él se pueden registrar requerimientos con descripción, prioridad, responsable y estado, clasificarlos entre funcionales y no funcionales, y ver cómo cada uno avanza desde que se define hasta que se implementa. También explicó la gestión de cambios, cuando se pide una modificación se evalúa el impacto, se actualiza y se valida, y todo queda registrado.

Práctica: Creó requerimientos y tareas, las relacionó con subtareas para organizar mejor el trabajo, hizo seguimiento del avance y mostró cómo el historial de cambios permite ver qué se modificó y qué elementos se vieron afectados.

Pérez Carlos: Habló de las ventajas de la herramienta, que es gratuita, que mejora la trazabilidad, que centraliza todo y que permite trabajar en equipo. También fue honesto con las limitaciones, la configuración inicial puede ser técnica, los usuarios necesitan capacitarse, y si el equipo no mantiene la información actualizada la herramienta pierde utilidad. Después relacionó Redmine con la IR y mencionó que guarda relación con estándares como SWEBOK, IEEE 29148 e ISO 25010.

Práctica: Registró horas de trabajo por actividad, incorporó integrantes al equipo, asignó requerimientos y tareas a cada uno, y mostró el diagrama de Gantt para visualizar el avance general del proyecto.

Sánchez Roselyn: Aplicó todo lo anterior a un proyecto concreto, un sistema de gestión para una camaronera. Identificó requerimientos funcionales como registrar el pesaje del camarón, calcular pagos automáticamente y consultar el historial de entregas, además de requerimientos no funcionales de seguridad y disponibilidad. Hizo la comparativa con Jira y Trello, destacando que Redmine es la única opción gratuita de las tres, y concluyó que la herramienta apoya al analista pero no lo reemplaza.

Práctica: Configuró los estados posibles de un requerimiento (Nueva, En análisis, Aprobada, En desarrollo, Cerrada), las prioridades (Baja, Normal, Alta, Urgente) y los tipos de petición (Requisito y Tarea), para mostrar cómo se gestiona el ciclo completo de un requerimiento.

Grupo F: REQI

Mayummi Trujillo: Explicó las ventajas de REQI, que tiene una interfaz bastante intuitiva y que no pide mayores conocimientos técnicos para usarla, que permite que todo el equipo trabaje con información actualizada al mismo tiempo, que tiene su propio marco metodológico que guía el proceso, y que sigue cada requerimiento hasta que se implementa, eso es lo que llaman trazabilidad. También mencionó que maneja un modelo freemium, lo cual lo hace accesible para proyectos con presupuesto limitado, y que tiene soporte para cadena de suministros.

Práctica: Mostró el diagrama del proyecto en REQI, con diferenciación de colores para distinguir actores y otros elementos del sistema.

Barrionuevo Carlos: Presentó REQI como una herramienta especializada en gestión de requerimientos. Explicó sus funcionalidades principales, gestión de requerimientos, visualización del sistema, trazabilidad de conformidad, colaboración en tiempo real, gestión de cadena de suministro, y un marco metodológico propio.

Práctica: Mostró cómo crear módulos en REQI, configurando el sistema padre, el nombre del sistema, la descripción, los responsables del módulo y quién lo va a revisar.

Mayummi Trujillo: Completó la parte práctica mostrando en detalle cómo se agregan requerimientos dentro de REQI, se llena el título, se elige el tipo de requerimiento, se define la fase en la que se integra, su nivel de criticidad, el responsable y la fuente, y en el campo de razón se escribe para qué sirve ese requerimiento, todo en texto. Al guardar, el ID se genera solo, de forma automática. También mostró cómo se configuran los criterios de aceptación de cada requerimiento, poniendo el título, la referencia, dónde está ubicado, en qué fase y en qué paquete, y aclaró que no es recomendable dejar un requerimiento en estado indefinido.

Práctica: Creó un proyecto en REQI con nombre y descripción, recorrió el dashboard para familiarizarse con la interfaz y mostró el proceso completo de registro de un requerimiento junto con su criterio de aceptación.

Quintero Erick: Habló de las limitaciones de REQI, que no tiene automatización con IA ni procesamiento de lenguaje natural, que sus integraciones con otras herramientas de desarrollo son bastante limitadas, que en proyectos de gran complejidad puede quedarse corta (funciona mejor para proyectos pequeños o medianos), y que la trazabilidad de cobertura de código, junto con la documentación y el soporte técnico, no son de lo más completo. Para cerrar esa parte presentó un caso de estudio de una camaronera, con el propósito de digitalizar el proceso de principio a fin.

Práctica: Fue el que más desarrolló la parte práctica del grupo. Creó el proyecto en REQI poniéndole nombre y descripción, recorrió el dashboard para ubicarse en la interfaz y entró al apartado de requerimientos. Ahí agregó un requerimiento completo: escribió el tipo de requerimiento (aclarando que no conviene dejarlo en indefinido), la fase del proyecto en la que se integra, su nivel de criticidad, quién queda como encargado, la fuente de donde viene ese requerimiento, y en razón explicó cuál es el propósito de tenerlo, todo redactado en texto. Mostró que al guardar, el sistema le genera solo el ID, sin que uno tenga que ponerlo a mano. Después configuró el criterio de aceptación de ese mismo requerimiento, definiendo bajo qué condición se considera aceptado, el título, la referencia, en qué ubicación y paquete se encuentra, y en qué fase del proyecto queda enmarcado.

Grupo G: Modern Requirements

Alvia Erick: Abrió la exposición con un dato fuerte, que el 70% de los defectos que llegan a producción vienen de errores en los requerimientos. Desde ahí presentó Modern Requirements como una plataforma empresarial que nació en 2006, se integró con Azure DevOps en 2018 y lanzó su módulo de IA en 2022. Explicó que cubre todo el ciclo de requerimientos y que maneja jerarquías de trabajo como Epics, Features, User Stories, Tasks y Bugs, y mencionó también el UseCase Suite para modelar casos de uso con plantillas de estándares conocidos.

Práctica: Mostró la interfaz de la herramienta con los sprints, las consultas y la trazabilidad, usando el editor con Querys y work item types, conectados visualmente mediante un plano cartesiano.

Arboleda Francisco: Explicó la parte de trazabilidad y revisión formal, que la herramienta permite crear matrices que conectan requerimientos con pruebas, código, defectos y diseño, en ambas direcciones. También habló del control de versiones con historial completo y gestión de baselines, de las firmas electrónicas que sirven para auditorías, y de ModernAI, que genera User Stories y analiza la calidad de los requerimientos. Además, mencionó el Simulation Suite, para hacer prototipos interactivos vinculados directamente a los requerimientos.

Práctica: Configuró un plan de entrega y asignó el sprint al equipo, con nombre, descripción, integrantes y criterios de campo con un límite de elementos.

Calle Kamila: Hizo la comparativa de Modern Requirements contra herramientas como DOORS, Jama y ReqView, destacando que Modern Requirements tiene IA generativa, trazabilidad completa y prototipado integrado, cosas que las otras no tienen o que tienen de forma parcial. Sus ventajas principales son la integración nativa con Azure DevOps, la documentación automática alineada a estándares y los flujos de aprobación con firmas electrónicas.

Práctica: Mostró el tablero de seguimiento, creó un requerimiento con título y descripción usando historias de usuario, luego creó una tarea vinculada a ese requerimiento con su tipo de enlace y descripción, y mostró cómo cambiar el estado de las actividades conforme avanzan.

Macias Jhostyn: Presentó las limitaciones, que la herramienta depende mucho del ecosistema de Microsoft, lo que la hace poco compatible con Jira o GitLab, que tiene un costo elevado para equipos pequeños, y que ModernAI necesita revisión humana cuando el dominio es muy técnico. También habló de cómo se podría aplicar en el proyecto del curso usando el plan gratuito de Azure DevOps.

Práctica: Mostró cómo crear sprints nuevos, asignar de qué trata cada uno y distribuir el trabajo entre los integrantes.

Grupo H: rmToo

Tigasi Paul: Arrancó con una pregunta que resume bien toda la exposición, ¿y si manejáramos los requerimientos con la misma seriedad que el código fuente? Explicó que cuando se trabaja con código se usa Git, hay historial de cambios y se puede ver quién cambió qué y cuándo, pero que cuando se escriben requerimientos normalmente se usa Word o un PDF, que no se versiona bien y que no se puede procesar de forma automática. De ahí viene la idea de Requirements as Code, donde el contenido en texto plano queda separado del formato, parecido a como en programación se separa la lógica de la interfaz.

Práctica: No vista

Romero Vaca: Explicó cómo funciona rmToo por dentro, los requerimientos se escriben en archivos de texto plano con extensión. req, donde se define el ID, la descripción, la prioridad, el estado y las dependencias de cada uno. Es ligero, se basa en texto plano y se integra con Git para llevar el control de versiones. La limitación más grande que mencionó es la curva de aprendizaje, porque todo se hace desde código y terminal, sin ninguna interfaz gráfica.

Práctica: No realizó demostración práctica.

Huilecapi Denisse: Habló de tres cosas puntuales, cómo rmToo detecta inconsistencias entre requerimientos, cómo se editan directamente los archivos y qué pasa en el sistema cuando se hace un cambio, y cómo Git permite comparar versiones para ver exactamente qué líneas cambiaron y por qué. Todo esto lo fue explicando a partir de una captura que tenía en la diapositiva.

Práctica: Mostró el funcionamiento de rmToo desde la terminal, ya sea cmd o PowerShell. En pantalla se veía un archivo real llamado sgu-auth.req, con campos como Description y Rationale.

rmToo marcaba en rojo el texto original y en verde el texto nuevo, antes de confirmar los cambios, todo por línea de comandos y sin ninguna interfaz gráfica.